

From pilot to production

The evaluation discipline that decides whether a model ships, or stalls

M Usman

An applied view of the gap every AI project has to cross.

Where this is heading

Building a pilot is easy now. Crossing to production is the test.

M Usman · From enterprise data to production AI.

The gap is not the model, it is the crossing

- The tooling has made a working pilot cheap. The hard part moved downstream: turning a notebook that scores well into a system that decides well, repeatably, and stays right.
- The crossing is where the value leaks out:
 - Only about **48%** of AI projects reach production, and a prototype takes about **8 months** to get there (Gartner, 2024)
 - At least **30%** of generative-AI projects are expected to be **abandoned after proof of concept** by the end of 2025 (Gartner, 2024)
 - **More than 80%** of AI projects fail, around twice the rate of non-AI IT projects (RAND, 2024)

A pilot proves a model can work once. Production asks whether it works honestly, every time, and whether anyone would notice if it stopped.

The slow step

A notebook score is a hypothesis, not a result

The pilot reports a number. The crossing asks four questions of that number, in order, and a model that cannot answer one does not advance: **Is the score honest? Does it map to a decision? Will it survive deployment? Will anyone catch it when it decays?**

The discipline that answers those four questions is evaluation, widened from a single metric into a sequence of gates. The rest of this deck is those gates.

M Usman · From enterprise data to production AI.

The crossing as a funnel



Pilots built

a notebook score

leaks: leakage and vanity metrics



Gate 1 • Honest evaluation

time-ordered split, leakage audit, honest metric

leaks: a metric that means nothing to a decision



Gate 2 • Decision value

metric tied to the cost of a wrong call, a real baseline

leaks: works on my machine



Gate 3 • Production-ready

train-serve parity, versioned API, reproducible rebuild

leaks: silent decay



Gate 4 • Live and watched

drift detection, ground-truth monitoring, retraining wired in

leaks: nobody notices



Sustained value

a model that decides well, repeatably, and stays honest

Each gate has one question, one common way to fail it, and one discipline that passes it. The next four slides take them in turn.

Gate 1 • Honest evaluation: is the score real?

The model's reported number, earned under a protocol that matches the data, so the score in the report is the score you will see in production.

Practical steps

- Split train, validation, test **first**; fit every transform on the train fold only
- Match the protocol to the data: stratified, grouped, or **time-ordered** splits
- Audit every feature for target and temporal leakage before trusting any result
- Resample and tune **inside** each fold; keep one untouched test set to the end

The pitfall: leakage and vanity metrics. Information from the future or the target slips into training; the score is spectacular and fictional, then collapses on real data. The only tell is a result too good to be true. Leakage is not a rare slip: one 2023 review found it in **294 papers across 17 scientific fields** (Kapoor and Narayanan).

What good looks like. The whole feature-and-resample chain sits inside the cross-validation loop. Treat a suspiciously high score as an alarm, not a win.

Illustrative tools: scikit-learn (Pipeline, StratifiedKFold, GroupKFold, TimeSeriesSplit) · imbalanced-learn · MLflow · SHAP.

Gate 2 · Decision value: does the score mean anything?

A model earns production only when its metric maps to a decision and the cost of getting that decision wrong.

Practical steps

- Pick the metric from the decision: precision, recall, F1, or PR-AUC, not raw accuracy
- Set the operating threshold where the business cost of a false positive meets a false negative
- State the baseline the model must beat: the current rule, the last model, or doing nothing
- Report the number with its uncertainty, not as a single hero figure

The pitfall: optimising the wrong number. A model tuned to a metric nobody decides on is a model nobody can use. High accuracy on a rare event can be worse than useless.

What good looks like. Every score is quoted against a baseline and a decision. "Better than what, and worth what?" has an answer before the model advances.

Illustrative practice: cost-sensitive thresholds · calibration · a named baseline per problem.

M Usman · From enterprise data to production AI.

Gate 3 • Production-ready: will it survive deployment?

The model packaged behind a versioned interface, reproducible away from the laptop it was trained on, so what shipped is what was evaluated.

Practical steps

- Package model, features, and dependencies together; pin versions; verify **train-serve parity**
- Reproduce the training run from source and data alone, on another machine
- Put the model behind a versioned API with a fast rollback path
- Release progressively: shadow, then canary, then full, never big-bang

The pitfall: works on my machine. A model that cannot be rebuilt, or that sees different features in serving than in training, is a result that does not travel. The skew is invisible until production decisions go wrong.

What good looks like. A clean rebuild from source reproduces the evaluated score, and the serving path is provably the training path.

Illustrative tools: FastAPI, BentoML • Docker • a model registry • feature parity checks.

Gate 4 • Live and watched: will anyone catch the decay?

The model in the decision path, monitored, so the day it stops being right is the day someone knows.

Practical steps

- Monitor three things: input **data drift**, **concept drift**, and **ground-truth performance**
- Close the loop: capture outcomes so live accuracy can be measured, not assumed
- Trigger retraining on a drift threshold or a measured drop, not the calendar alone
- Keep the evaluation harness in production, not only in the notebook it was born in

The pitfall: silent decay. A shipped model is a depreciating asset; the world drifts, the model quietly stops working, and the first alarm is a business metric moving for reasons nobody can name.

What good looks like. Monitoring is part of the build. A model is not "done" until drift detection, performance tracking, and a retraining trigger are wired in.

Illustrative tools: Evidently, Arize, NannyML • a feature store • scheduled re-evaluation.

M Usman • From enterprise data to production AI.

The evaluation-maturity ladder

5 · Gated and self-correcting

every model passes the four gates to ship; drift and performance monitored; retraining automated; a sign-off gate

4 · Served and watched

leakage-safe model behind a versioned API; canary release; drift and ground-truth monitoring in place

3 · Evaluated honestly

protocol matches the data; no leakage; metric tied to a decision and a baseline; reproducible run

2 · Scored once

a single split, accuracy reported, no baseline, no leakage audit; the number is taken on trust

1 · It works in the notebook

one run, one laptop; nothing reproduced, nothing questioned

Most stalled pilots sit at rung 2: a real number, taken on trust, that never faces a gate. The climb to rung 5 is the crossing.

M Usman · From enterprise data to production AI.

What good looks like, in one rule

A model does not advance to the next gate until it passes the one it is on. The discipline is not extra work bolted on at the end; it is the order the work is done in.

- **Honest before impressive.** A leakage audit comes before the celebration.
- **Decision before metric.** The number is chosen from the call it informs, against a named baseline.
- **Reproducible before shipped.** What ships is what was evaluated, provably.
- **Watched before trusted.** Monitoring is wired in at build, not after the first failure.

M Usman · From enterprise data to production AI.

The lens behind this view

My doctoral work is on efficient, proactive AI, where honest evaluation and the cost of a wrong prediction are not a formality, they are the whole problem. A failure predicted too late, or a false alarm raised too often, both carry a real price. That is why the discipline of measuring a model honestly is the discipline I work in.

That same discipline is public and worked, not asserted. Across four live repositories I evaluate on time-ordered and grouped protocols, catch a leakage trap and report the honest, lower number, and serve a model behind a versioned interface. Expressing that at production scale, with full drift monitoring and automated retraining, is the part I am building now.

The four worked examples, with code-reproduced numbers, are public: github.com/mtusman-ai

M Usman · From enterprise data to production AI.

In one line

AI does not stall because the model is weak; it stalls in the crossing from pilot to production. Make the score honest, tie it to a decision, ship what you evaluated, and watch it for decay. The pilot that scored well becomes a system that decides well.

M Usman · From enterprise data to production AI.

Sources

Gartner (May 2024): 48% of AI projects reach production, ~8 months prototype-to-production · Gartner (2024): $\geq 30\%$ of generative-AI projects abandoned after proof of concept by end-2025 · RAND, *The Root Causes of Failure for AI Projects* (2024): 80%+ fail · Evaluation and MLOps practice draws on the ML-evaluation and reproducibility literature: leakage and protocol design; Google *Rules of ML* and *ML Test Score*; data- and model-drift monitoring. Kapoor and Narayanan, *Leakage and the Reproducibility Crisis in Machine-Learning-based Science* (Patterns, Cell Press, 2023): leakage found in 294 papers across 17 scientific fields.

M Usman · From enterprise data to production AI. · mtusman-ai.github.io/M.Usman